Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. doi: 10.18637/jss.v000.i00

exdqlm: An R Package for Estimation and Analysis of Flexible Dynamic Quantile Linear Models

Antonio Aguirre
University of California Santa Cruz

Raquel Barata
University of California Santa Cruz

Raquel Prado
University of California Santa Cruz

Bruno Sansó
University of California Santa Cruz

Abstract

We present the R package **exdqlm** as a tool for dynamic quantile regression. The main focus of the package is to provide a framework for Bayesian inference and forecasting of flexible dynamic quantile linear models (exDQLMs). exDQLMs utilize a new family of error distributions for parametric quantile regression, the extended asymmetric Laplace (exAL), a generalization of the asymmetric Laplace (AL) which is commonly used in parametric quantile regression. Estimation of a dynamic quantile linear model, which utilizes the AL, is included in the package as a special case. Non-time-varying quantile regression models are also a special case of our methods. Further, routines for estimation of a nonlinear relationship between the response and a given input variable at a specified quantile via a transfer function model are available. The software provides exact posterior inference via Markov chain Monte Carlo (MCMC) and fast approximate posterior inference via Laplace delta variational Bayes (LDVB). While MCMC provides sample-based exploration of the posterior distribution, the approximations obtained using the LDVB algorithm enable fast inference for long time series at a fraction of the memory and computational costs of the MCMC. A routine for forecasting the dynamic quantile is available in the package, as well as several quantitative and visual diagnostics for model evaluation. We illustrate the implementation of the functions and algorithms in the **exdqlm** package with a step-by-step guide for the analysis of several real data sets.

Keywords: dynamic quantile regression, dynamic models, asymmetric Laplace, variational Bayes, coordinate ascent variational inference, quantile regression.

1. Introduction

The purpose of the R (R Core Team 2013) package **exdqlm** is to provide an accessible and comprehensive tool for flexible Bayesian estimation of a single dynamic quantile. Quantile regression is an area that has gained popularity in the non-time-varying setting as a robust alternative to traditional mean-centric approaches. However, time-varying methods and software implementations remain limited. In a Bayesian setting, parametric quantile regression models are almost exclusively based on the asymmetric Laplace (AL) likelihood, as it corresponds to the check loss function that is minimized in traditional quantile regression problems (Yu and Moyeed 2001; Koenker 2005). A mixture representation of the AL provides computational advantages for the exploration of the posterior distribution of the model parameters. This has contributed to its popularity. However the AL is known to have several shortcomings in terms of flexibility. Very general error distributions for quantile regression have been considered extensively in the Bayesian non-parametric literature (Kottas and Krnjajić 2009; Reich, Bondell, and Wang 2009; Taddy and Kottas 2010), but the computational cost of these methods do not scale well to dynamic models for long times series. Yan and Kottas (2017) present a parametric extension of the AL, the extended asymmetric Laplace (exAL), which overcomes the restrictive aspects of the distribution while preserving its computational advantages. Barata, Prado, and Sansó (2022) utilize the new family of error distributions presented in Yan and Kottas (2017) to develop a flexible dynamic quantile linear model (exDQLM). This is the model implemented in the package **exdqlm**.

Dynamic quantile models typically consist of a set of variables that summarize the properties of the quantile of interest and a structure that defines the evolution of these variables, which together represent the behavior of a quantile over time. Our package **exdqlm** is a powerful and versatile tool for dynamic quantile modeling. The model form is robust enough to estimate any quantile regardless of non-standard features in the data distribution (i.e., skewness or extreme observations). Further, relevant features such as long-term trends or seasonality are easily incorporated into the evolution of the quantile. The package includes both a Markov chain Monte Carlo (MCMC) algorithm for exact posterior inference and a Laplace delta variational Bayes (LDVB) algorithm for fast, yet accurate approximate estimation. A legacy importance sampling variational Bayes (ISVB) routine is retained in the package for backward compatibility, however in this paper we focus on LDVB as the primary approximation. The LDVB routine makes our methods accessible to applications with very large datasets and supports quick model exploration and selection. The exDQLM framework facilitates k-step-ahead forecasting for the estimated quantile, also implemented in the package. Further, a transfer function extension is included in the software as a straightforward method to quantify a nonlinear relationship between a response and a given input variable at a specified quantile. Several visual and quantitative model diagnostics are provided to support model examination. Finally, MCMC and LDVB algorithms for static quantile regression using the exAL error distribution are also included in the package. The package is protected under the MIT License, and is available from the Comprehensive R Archive Network (CRAN) at `https://CRAN.R-project.org/package=exdqlm`.

Well-developed software implementations of non-time-varying quantile regression models which utilize the AL are available in the Bayesian R package **bayesQR** (Benoit, Al-Hamzawi, Yu, and Van den Poel 2011) and, equivalently, a classical version, which utilizes the check-loss function (Koenker 2005), is available in the package **quantreg** (Koenker 2021). As a result of

the AL being a special case of the exAL, the methods used in the Bayesian package **bayesQR** are also a special case of our exDQLM implemented in **exdqlm**. Further, there are many established packages for dynamic modeling including **dlm** (Petris and Gilks 2018) and **dynr** (Ou, Hunter, Chow, Ji, Chen, Hung, Lee, Li, and Park 2021), however, implementations of dynamic quantile modeling are much more limited. To our knowledge, the package **quantreg** includes a routine which allows for a time-evolving structure in the regression coefficients (a special case of the routines in our package), but this is the extent of the software available for dynamic quantile regression.

In the remainder of this paper we introduce the package as follows. In Section 2, we describe the modeling framework, including the algorithms used for efficient posterior inference and our methods for including nonlinear relationships. We also discuss static quantile regression as a special case of the framework. In Section 3 we detail the model diagnostics and forecasting method implemented in the package. Then, in Section 4, we walk through four step-by-step examples on how to use the package. Finally, in Section 5, we present a summary of the package capabilities.

2. Extended dynamic quantile linear models

Consider a set of scalar time-evolving responses, y_t, for times $t = 1, \ldots, T$. For each t, an extended dynamic quantile linear model (exDQLM) for inference on a single p_0-th quantile is defined by

$$\text{Observation equation:} \quad y_t = \mathbf{F}_t' \boldsymbol{\theta}_t + \epsilon_t, \qquad \epsilon_t \sim \text{exAL}_{p_0}(0, \sigma, \gamma) \tag{1}$$

$$\text{System equation:} \quad \boldsymbol{\theta}_t = \mathbf{G}_t \boldsymbol{\theta}_{t-1} + \boldsymbol{\omega}_t, \qquad \boldsymbol{\omega}_t \sim \text{N}(\mathbf{0}, \mathbf{W}_t). \tag{2}$$

Here $\mathbf{F}_t$ is the $q \times 1$ regression vector of the covariates corresponding to the $q \times 1$ parameter vector $\boldsymbol{\theta}_t$ at time t, $\mathbf{G}_t$ is the $q \times q$-dimensional evolution matrix defining the structure of the parameter vector evolution in time, and the normal distribution according to which the system error $\boldsymbol{\omega}_t$ evolves is q-variate. The observational errors ϵ_t of the exDQLM follow an exAL distribution for fixed quantile p_0, whose density is denoted as $\text{exAL}_{p_0}(\cdot)$, and it is such that $\int_{-\infty}^{0} \text{exAL}_{p_0}(\epsilon_t|0, \sigma, \gamma) \text{d}\epsilon_t = p_0$. Thus, the observation equation in Equation (1) implies that $\mathbf{F}_t' \boldsymbol{\theta}_t$ corresponds to the p_0-quantile of y_t, even if this is not made explicit in the notation above. Table 1 shows the function parameters that define the structure of the dynamic quantile, specifically $\mathbf{F}_t$ and $\mathbf{G}_t$ for $t = 1, \ldots, T$.

The exAL distribution is constructed from an extension of the location-scale mixture representation of the AL (Kozumi and Kobayashi 2011), details of which can be found in Yan and Kottas (2017). This results in a mixture representation of $\text{exAL}_{p_0}(0, \sigma, \gamma)$ as follows:

$$\int \int_{\mathbb{R}^+ \times \mathbb{R}^+} \text{N}(y_t | C(p, \gamma)\sigma|\gamma| s_t + A(p) v_t, \sigma B(p) v_t) \text{Exp}(v_t|\sigma) \text{N}^+(s_t|0, 1) \text{d}v_t \text{d}s_t. \tag{3}$$

Here, $p = p(p_0, \gamma) = I(\gamma < 0) + \{[p_0 - I(\gamma < 0)]/g(\gamma)\}$ where $g(\gamma) = 2\Phi(-|\gamma|)\exp(\gamma^2/2)$, and $\Phi(\cdot)$ denotes the standard normal CDF. $A(p)$, $B(p)$, $C(p, \gamma)$ are the functions of p and γ: $A(p) = \frac{1-2p}{p(1-p)}$, $B(p) = \frac{2}{p(1-p)}$, $C(p, \gamma) = [I(\gamma > 0) - p]^{-1}$. Lastly, $\text{Exp}(v|\sigma)$ denotes the exponential distribution with mean σ, and $\text{N}^+(s_t|0, 1)$ denotes a normal distribution truncated to the positive reals with mean 0 and variance 1. This mixture representation of the exAL

Input list	`model`				`PriorSigma`		`PriorGamma`		
Symbol	$\mathbf{F}_{1:T}$	$\mathbf{G}_{1:T}$	$\mathbf{m}_0$	$\mathbf{C}_0$	a_σ	b_σ	m_γ	s_γ	ν_γ
List component	`FF`	`GG`	`m0`	`C0`	`a_sig`	`b_sig`	`m_gam`	`s_gam`	`df_gam`
Default	-	-	$\mathbf{0}_q$	$1e3\mathbf{I}_q$	2.1	1.1	0	1	1

Table 1: Translation from mathematical symbols to parameters used by **exdqlm** functions, as well as default values where applicable. $\mathbf{0}_q$ denotes a q-dimensional vector of zeros, and $\mathbf{I}_q$ denotes the q-dimensional idenity matrix.

can be exploited to rewrite the exDQLM as the following hierarchical model for $t = 1, \ldots, T$:

$$y_t|\boldsymbol{\theta}_t, \sigma, \gamma, v_t, s_t \sim \mathrm{N}(y_t|\mathbf{F}_t'\boldsymbol{\theta}_t + C(p,\gamma)\sigma|\gamma|s_t + A(p)v_t, \sigma B(p)v_t) \tag{4}$$

$$v_t, s_t|\sigma \sim \mathrm{Exp}(v_t|\sigma)N^+(s_t|0,1) \tag{5}$$

$$\boldsymbol{\theta}_t|\boldsymbol{\theta}_{t-1}, \mathbf{W}_t \sim \mathrm{N}(\mathbf{G}_t\boldsymbol{\theta}_{t-1}, \mathbf{W}_t). \tag{6}$$

This hierarchical form is leveraged for the posterior inference implemented within the **exdqlm** package.

2.1. Prior specification

Under a Bayesian formulation and given p_0, we specify priors for the initial state vector $\boldsymbol{\theta}_0$, scale parameter σ and skewness parameter γ. For $\boldsymbol{\theta}_0$, we assume a q-variate conjugate normal prior $\boldsymbol{\theta}_0 \sim \mathrm{N}(\mathbf{m}_0, \mathbf{C}_0)$. For σ, we assume a conjugate inverse-gamma prior denoted as $\mathrm{IG}(a_\sigma, b_\sigma)$. In many cases a strong prior on σ is necessary to guarantee fast posterior convergence and estimation. This is illustrated in the examples of Section 4. The parameter γ has bounded support over the interval (L, U) where L is the negative root of $g(\gamma) = 1 - p_0$ and U is the positive root of $g(\gamma) = p_0$ (Yan and Kottas 2017). In contrast to the flat prior suggested by Yan and Kottas (2017), we find that using a proper prior on the skewness parameter promotes reliable posterior inference. Thus, we place a Student-t distribution truncated to the interval (L, U) as the prior for γ, i.e. $\gamma \sim \mathrm{t}_{(L,U)}(m_\gamma, s_\gamma)$ with ν_γ degrees of freedom. The prior parameters used in the **exdqlm** functions, their corresponding mathematical symbols, and default values are shown in Table 1.

2.2. Posterior estimation

The hierarchical representation of the exDQLM facilitates posterior simulation using Markov Chain Monte Carlo (MCMC) algorithms. This is implemented in the function `exdqlmMCMC()`. Gibbs sampling steps are used to update the sets of latent variables s_t and v_t for $t = 1, \ldots, T$. The dynamic regression coefficients are sampled using a forward filtering backwards sampling algorithm (Carter and Kohn 1994; Frühwirth-Schnatter 1994). The scale and skewness parameters are then updated separately from the latent-state blocks. By default, the package uses a slice-sampling update for the (σ, γ) block, drawing σ from its full conditional distribution and updating γ with a bounded univariate slice sampler. Alternatively, the package also provides joint random-walk Metropolis-Hastings (MH) kernels on transformed coordinates, including a Laplace-informed proposal covariance that can be refreshed during burn-in. When desired, the chain can be warm-started from either the LDVB or legacy ISVB approximation; this initialization is separate from the subsequent MCMC proposal kernel. An example

exdqlm function	`exdqlmMCMC()`				`exdqlmLDVB()`	
Symbol	$\mathcal{K}_{\sigma,\gamma}$	Σ_{MH}	N_{BURN}	N_{MCMC}	ϵ_{tol}	N_{SAMP}
Input	`mh.proposal`	`Sig.mh`	`n.burn`	`n.mcmc`	`tol`	`n.samp`
Default	`"slice"`	$0.05\mathbf{I}_2$	2000	1500	0.1	200

Table 2: Translation from mathematical symbols to parameters used in functions `exdqlmMCMC()` and `exdqlmLDVB()`, as well as default values.

of this functionality can be found in Section 4.1. We perform a total of $N_{BURN} + N_{MCMC}$ MCMC iterations, discarding the first N_{BURN} and retaining the remaining N_{MCMC} posterior samples. The object created by `exdqlmMCMC()` includes posterior samples of all parameters $(\boldsymbol{\theta}_{1:T}, v_{1:T}, s_{1:T}, \gamma, \sigma)$, filtered and smoothed summaries for the state vector $\boldsymbol{\theta}_{1:T}$, and diagnostics for the proposal kernel used in the (σ, γ) update.

For purposes of model selection or large time series data sets, posterior sampling with the MCMC algorithm can be computationally demanding. To this end, we include in the package a Laplace delta variational Bayes (LDVB) routine, `exdqlmLDVB()`, for fast and accurate approximate posterior estimation. The variational Bayes (VB) framework approximates the posterior distribution with partitioned variational distributions (Ostwald, Kirilina, Starke, and Blankenburg 2014; Beal 2003). Under a mean-field factorization, this induces a blockwise coordinate-ascent variational inference scheme. For the exDQLM, the blocks associated with the dynamic coefficients and latent variables admit recognizable closed-form updates, while the joint variational distribution of σ and γ is non-conjugate. Following the non-conjugate VB strategy of Wang and Blei (2013), we approximate this block with a Laplace approximation on transformed coordinates and then use a second-order Delta method to evaluate the expectations of functions of (σ, γ) needed by the remaining variational updates. Thus, the mean-field factorization induces the blockwise CAVI updates, while the non-conjugate (σ, γ) block is handled through the Laplace-Delta approximation. The dynamic regression coefficients are updated within each iteration using a forward filtering backwards smoothing algorithm. Convergence is assessed jointly through the state, scale, skewness, and ELBO diagnostics. After the algorithm reaches convergence with tolerance ϵ_{tol}, N_{SAMP} samples are drawn from the variational distributions, resulting in an approximate sample of the posterior distribution. The output of `exdqlmLDVB()` includes the approximate posterior samples as well as the parameters of the variational distributions for all parameters $(\boldsymbol{\theta}_{1:T}, v_{1:T}, s_{1:T}, \gamma, \sigma)$. For the (γ, σ) block, the return also contains the transformed mode, approximate covariance matrix, derived moments, and ELBO terms associated with the Laplace-Delta approximation. The package also retains a legacy importance-sampling VB routine, `exdqlmISVB()`, but in this paper we restrict attention to MCMC and LDVB as the primary inferential tools.

Table 2 shows the input parameters of the fitting functions `exdqlmMCMC` and `exdqlmLDVB`, their mathematical symbols, and their default values.

The argument `Sig.mh` is only used when `mh.proposal` selects a joint random-walk MH kernel; it is not used under the default slice-sampling update.

2.3. Specification of the evolution covariance via discount factors

Both estimation algorithms, `exdqlmMCMC` and `exdqlmLDVB`, utilize discount factors to specify the time-evolving covariance matrices $\mathbf{W}_t$ introduced in Equation (2) (West, Harrison, and

Migon 1985). More precisely, we define the evolution variance matrix $\mathbf{W}_t = \frac{1-\delta}{\delta}\mathbf{G}_t\mathbf{C}_{t-1}\mathbf{G}_t'$ where $\mathbf{C}_{t-1}$ denotes the posterior variance of the state vector $\boldsymbol{\theta}_{t-1}$ at time $t-1$ and δ is a discount factor such that $0 < \delta \leq 1$. The discount factor δ can be specified by the user via function parameter `df`. Selection of discount factors is typically done by optimizing a model-checking criterion. We suggest using the KL divergence or posterior predictive loss criterion discussed in Section 3. A brief example of this method for discount factor selection can be found in Section 4.2.

Discounting by component of the state-space model is also a functionality of our estimation algorithms. Suppose the state vector is comprised of h components $\boldsymbol{\theta}_{it}$ of dimension q_i for $i = 1, \ldots, h$ such that $\boldsymbol{\theta}_t' = (\boldsymbol{\theta}_{1t}', \ldots, \boldsymbol{\theta}_{ht})'$ and $\sum_{i=1}^{h} q_i = q$. If $\mathbf{W}_{it}$ denotes the ith component of the evolution covariance matrix such that $\mathbf{W}_t = \text{blockdiag}\{\mathbf{W}_{1t}, \ldots, \mathbf{W}_{ht}\}$ and δ_i denotes a discount factor for the ith component, we can define the evolution variance matrix by component as $\mathbf{W}_{it} = \frac{1-\delta_i}{\delta_i}\mathbf{G}_{it}\mathbf{C}_{i,t-1}\mathbf{G}_{it}'$. Here $\mathbf{G}_{it}$ and $\mathbf{C}_{i,t-1}$ denote the ith components of $\mathbf{G}_t$ and $\mathbf{C}_{t-1}$, respectively. The function parameters `df` and `dim.df` allow users to specify a set of component discount factors $(\delta_1, \ldots, \delta_h)$ and their dimensions $(q_1, \ldots, q_h)$, respectively, within the state-space model. Examples of this feature can be found in Section 4. For a full review of discount factors including discounting by component, see West *et al.* (1985).

2.4. Transfer function model

Quantifying the relationship between an input and a quantile of a response is a non-trivial task, particularly when the relationship is nonlinear. In mean-centric dynamic modeling, transfer functions are a straightforward method to incorporate variables which measure the combined effect of current and past regression effects (West *et al.* 1985). An extension of the transfer function approach to the dynamic quantile modeling setting is implemented within the **exdqlm** package, particularly for the LDVB algorithm.

For $t = 1, \ldots, T$ and univariate input, X_t, a transfer function exDQLM with exponential decay is defined by

$$y_t|\boldsymbol{\theta}_t, \gamma, \sigma \sim \text{exAL}_{p_0}(\mathbf{F}_t'\boldsymbol{\theta}_t + \zeta_t, \sigma, \gamma) \tag{7}$$

$$\boldsymbol{\theta}_t|\boldsymbol{\theta}_{t-1}, \mathbf{W}_t \sim \text{N}(\mathbf{G}_t\boldsymbol{\theta}_{t-1}, \mathbf{W}_t) \tag{8}$$

$$\zeta_t|\zeta_{t-1}, \psi_{t-1}, \omega_t \sim \text{N}(\lambda\zeta_{t-1} + X_t\psi_{t-1}, \omega_t) \tag{9}$$

$$\psi_t|\psi_{t-1}, \nu_t \sim \text{N}(\psi_{t-1}, \nu_t). \tag{10}$$

Here ζ_t captures the effect of the current and past regression effects on the quantile at time t. Alternatively, the parameter ψ_t determines the instantaneous effect X_t has on the quantile. The parameter $\lambda \in [0, 1]$ represents the memory of the regression effect up to time t. This effect decays at an exponential rate, reducing by a factor of λ at every time step. More explicitly, the effect of X_t on the quantile at time $t + k$ is $\lambda^k\psi_{t-1}X_t$. Thus, the series $k_t \geq \frac{\log(\epsilon)-\log(|\psi_{t-1}X_t|)}{\log(\lambda)}$ represents a lower bound for the number of time steps until the effect of X_t is less than or equal to a fixed ϵ. The median value of k_t corresponding to $\epsilon = 1\text{e}{-3}$ is included as an output of our transfer function routine.

Conditional on a fixed λ, say $\tilde{\lambda}$, we augment the LDVB algorithm to incorporate the transfer function structure by simply replacing $\mathbf{F}_t, \boldsymbol{\theta}_t, \mathbf{G}_t$, and $\mathbf{W}_t$ in Equations (1)-(2) with $\tilde{\mathbf{F}}_t' = (\mathbf{F}_t', 1, 0)$, $\tilde{\boldsymbol{\theta}}_t' = (\boldsymbol{\theta}_t', \zeta_t, \psi_t)$, $\tilde{\mathbf{G}}_t = \text{blockdiag}\left\{\mathbf{G}_t, \begin{pmatrix} \tilde{\lambda} & X_t \\ 0 & 1 \end{pmatrix}\right\}$, and $\tilde{\mathbf{W}}_t = \text{blockdiag}\left\{\mathbf{W}_t, \begin{pmatrix} \omega_t & 0 \\ 0 & \nu_t \end{pmatrix}\right\}$, respectively. This augmentation is implemented in the function `transfn_exdqlmLDVB()`, an

Symbol	X_t	$\tilde{\lambda}$	$(\delta_\zeta, \delta_\psi)$	$\mathbf{m}_0^{tf}$	$\mathbf{C}_0^{tf}$
`transfn_exdqlmLDVB()` input	`X`	`lam`	`tf.df`	`tf.m0`	`tf.C0`

Table 3: Translation from mathematical symbols to parameters used in function `transfn_exdqlmLDVB()`.

extension of the routine found in `exdqlmLDVB()`.

Similar to the specification of $\mathbf{W_t}$ discussed in Section 2.3, discount factors $(\delta_\zeta, \delta_\psi)$ are utilized to specify $\left(\begin{smallmatrix} \omega_t & 0 \\ 0 & \nu_t \end{smallmatrix}\right)$. A bivariate normal conjugate prior on $(\zeta_0, \psi_0)' \sim N(\mathbf{m}_0^{tf}, \mathbf{C}_0^{tf})$ completes the transfer function model extension. Table 3 shows the additional parameters used in `transfn_exdqlmLDVB()`, as well as their mathematical symbols discussed in this section.

2.5. Special cases, including static quantile regression

The first special case of our exDQLM arises from the fact that the exAL is an extension of the AL (Yan and Kottas 2017). When $\gamma = 0$, the observational error ϵ_t in Equation (1) reduces such that $\epsilon_t \sim \text{exAL}_{p_0}(\epsilon_t|0, \sigma, \gamma) = \text{AL}_{p_0}(\epsilon_t|0, \sigma)$. This special case with observational errors distributed independently from an AL is the dynamic quantile linear model (DQLM) presented in Gonçalves, Migon, and Bastos (2017). The DQLM can be implemented with our package using the parameter `dqlm.ind = TRUE`.

Non-time-varying quantile regression is also a special case of our methodology. More specifically, when discount factors are set to 1, the evolution covariance $\mathbf{W}_t$ reduces to $\mathbf{0}$ resulting in non-time-varying parameters (West *et al.* 1985). Thus, the model reduces to the static exAL regression methodology of Yan and Kottas (2017). This model can be implemented using our functions `exal_static_mcmc()` or `exal_static_LDVB()`, which implement MCMC and LDVB algorithms, respectively.

The final special case is the intersection of the previous two. A static quantile regression model with observational errors distributed according to an AL corresponds to Bayesian linear quantile regression under the AL working likelihood, first presented in Yu and Moyeed (2001). This methodology, utilized to create the Bayesian package for quantile regression **bayesQR**, can also be implemented with our package using functions `exal_static_mcmc()` or `exal_static_LDVB()` with parameter `dqlm.ind = TRUE`, which acts as a convenience flag fixing $\gamma = 0$.

Examples of these special cases can be found in Section 4. A static quantile regression example specifically can be found in Section 4.4. For further details on the methodology outlined in this section, see Barata *et al.* (2022).

3. Model Diagnostics and Forecasting

To evaluate the quantile inference and predictive performance of the exDQLM, several quantitative and visual model diagnostics are included in the package.

The one-step-ahead predictive distribution function introduced by Rosenblatt (1952) is a useful diagnostic for models in which the marginal forecast distributions are unrecognizable or difficult to compute (Huerta, Jiang, and Tanner 2003; Prado, Molina, and Huerta 2006).

For the exDQLM, this distribution is defined as

$$u_t = \Pr(Y_t \leq y_t | y_{1:t-1}, v_{1:T}, s_{1:T}, \sigma, \gamma), \tag{11}$$

with $y_{1:t-1} = y_1, \ldots, y_{t-1}$, $v_{1:T} = (v_1, \ldots, v_T)$ and $s_{1:T} = (s_1, \ldots, s_T)$. Here u_t defines an independent sequence which is uniformly distributed on the interval $(0, 1)$ (Rosenblatt 1952). Conditional on $\{v_{1:T}, s_{1:T}, \sigma, \gamma\}$, the forecast distribution of y_t within the exDQLM is normally distributed. Therefore, conditional on MAP estimates $\{\hat{v}_{1:T}, \hat{s}_{1:T}, \hat{\sigma}, \hat{\gamma}\}$ we estimate the sequence as

$$\hat{u}_t = \Phi(y_t | y_{1:t-1}, \hat{v}_{1:T}, \hat{s}_{1:T}, \hat{\sigma}, \hat{\gamma}) \tag{12}$$

where Φ denotes the normal CDF.

Using this estimated sequence $\{\hat{u}_t\}$, we can assess the model performance in several ways. First, we visually examine the correlation of the estimated sequence via an ACF plot. Second, by transforming the values with a standard normal inverse CDF, we compare their distributional shape to that of a standard normal with a normal QQ-plot. To quantify the divergence of this transformed sequence from normality we compute the KL divergence (Kullback and Leibler 1951), $\mathrm{KL}(h, \phi) = \int_{-\infty}^{\infty} h(x) \log \frac{h(x)}{\phi(x)} \mathrm{d}x$, where h denotes the numerically approximated density of our transformed sample and ϕ is the standard normal density. A smaller KL divergence suggests a superior model fit. Lastly, the MAP standardized forecast errors,

$$\frac{y_t - \mathsf{E}[y_t | y_{1:t-1}, \hat{v}_{1:T}, \hat{s}_{1:T}, \hat{\sigma}, \hat{\gamma}]}{\sqrt{\mathrm{Var}(y_t | y_{1:t-1}, \hat{v}_{1:T}, \hat{s}_{1:T}, \hat{\sigma}, \hat{\gamma})}} \tag{13}$$

can also be used as an additional graphical aid in examining the distribution at specific time points. These MAP standardized forecast errors are included in the output of the `exdqlmMCMC` and `exdqlmLDVB` functions.

For a final diagnostic, we include Gelfand and Ghosh (1998) posterior predictive loss criterion (pplc) with check loss function $\rho_{p_0}(x) = x[p_0 - I(x < 0)]$. That is,

$$\text{pplc} = \sum_t \mathsf{E}[\rho_{p_0}(y_t^{obs} - y_t^{rep}) | y_{1:T}], \tag{14}$$

where the expectation is with respect to the posterior predictive distribution of y_t, $p(y_t^{rep} | y_{1:T})$. This distribution is used to generate a replicate dataset and is the expected value of the observation equation (1) with respect to the posterior distributions of the parameters. Again, a smaller pplc value suggest a superior model fit. Samples from the posterior replicate distributions are included in the output of the `exdqlmMCMC()` and `exdqlmLDVB()` functions.

The function `exdqlmDiagnostics()` implements the model checking criteria discussed in this section. When an output from either `exdqlmMCMC()` and `exdqlmLDVB()` is passed to `exdqlmDiagnostics()`, the resulting object includes: the estimated sequence $\{\hat{u}_t\}$, the KL divergence, pplc, the ordered pairs of the QQ-plot, and autocorrelations by lag. By default, the function also produces the QQ-plot, ACF plot and MAP standard forecast error plot. Examples of the utility of `exdqlmDiagnostics()` can be found in the following Section 4.

3.1. k-step-ahead forecast

For an arbitrary time $\tilde{t}$, the k-step-ahead future marginal distribution of the quantile is

$$\mathbf{F}'_{\tilde{t}+k} \boldsymbol{\theta}_{\tilde{t}+k} | y_1, \ldots, y_{\tilde{t}} \sim \mathrm{N}(\mathbf{F}'_{\tilde{t}+k} a_{\tilde{t}}(k), \mathbf{F}'_{\tilde{t}+k} \mathbf{R}_{\tilde{t}}(k) \mathbf{F}_{\tilde{t}+k}) \tag{15}$$

Symbol	$\tilde{t}$	k	$\mathbf{F}_{(\tilde{t}+1):(\tilde{t}+k)}$	$\mathbf{G}_{(\tilde{t}+1):(\tilde{t}+k)}$
`exdqlmForecast()` input	`start.t`	`k`	`fFF`	`fGG`

Table 4: Translation from mathematical symbols to additional parameters used in function `exdqlmForecast()`.

Function	Output	Description
Model construction		
`polytrendMod()`	`exdqlm`	Create n-th order polynomial component.
`seasMod()`	`exdqlm`	Create Fourier form periodic component.
`regMod()`	`exdqlm`	Create standard regression component.
`as.exdqlm()`	`exdqlm`	Manipulates `list` into `exdqlm` object.
Model fitting – Input `exdqlm` objects		
`exdqlmMCMC()`	`exdqlmMCMC`	Estimate the posterior with MCMC algorithm.
`exdqlmLDVB()`	`exdqlmLDVB`	Estimate the posterior with LDVB algorithm.
`transfn_exdqlmLDVB()`	`exdqlmLDVB`	Estimate transfer function exDQLM using LDVB.
Model fit exploration – Input `exdqlmMCMC` or `exdqlmLDVB` objects		
`exdqlmForecast()`	`exdqlmForecast`	Compute k-step-ahead forecast quantiles.
`exdqlmDiagnostics()`	`exdqlmDiagnostic`	Compute & plot model diagnostics.
`exdqlmPlot()`	`list`	Plot posterior summary of dynamic quantile.
`compPlot()`	`list`	Plot posterior summary of component/parameter.

Table 5: Overview of functions available for dynamic quantile regression in **exdqlm**. Generic S3 methods defined for the objects listed in the table include `plot()`, `print()`, and `summary()`.

where $\mathbf{a}_{\tilde{t}}(k) = \mathbf{G}_{\tilde{t}+k}\mathbf{a}_{\tilde{t}}(k-1)$, $\mathbf{R}_{\tilde{t}}(k) = \mathbf{G}_{\tilde{t}+k}\mathbf{R}_{\tilde{t}}(k-1)\mathbf{G}'_{\tilde{t}+k} + \mathbf{W}_{\tilde{t}+k}$, $\mathbf{a}_{\tilde{t}}(0) = \mathbf{m}_{\tilde{t}}$, and $\mathbf{R}_{\tilde{t}}(0) = \mathbf{C}_{\tilde{t}}$, with $\mathbf{m}_{\tilde{t}}$ and $\mathbf{C}_{\tilde{t}}$ denoting the filtered mean and covariance of $\boldsymbol{\theta}_{\tilde{t}}$, respectively. This distribution is implemented in the function `exdqlmForecast` which is compatible with the outputs from both the MCMC and LDVB algorithms. Table 4 shows the input parameters used in the function, as well as their mathematical symbols seen in this section. The output of `exdqlmForecast()` includes the parameters of the forecasted distribution as well as $\mathbf{a}_{\tilde{t}}(k)$ and $\mathbf{R}_{\tilde{t}}(k)$ for the k steps.

Legacy ISVB routines remain available in the package for backward compatibility, but they are omitted from the primary workflow summary in Table 5.

4. Examples

Table 5 provides an overview of the **exdqlm** functions and the resulting objects. We demonstrate the general workflow and key features of the package through several analyses on real datasets. The following examples are implemented on a personal computer with an Apple M1 processor. We begin by loading the **exdqlm** package used for all examples.

```
R> library(exdqlm)
```

4.1. Lake Huron

In this example, we consider the Lake Huron time series from the package **datasets**. The data are annual measurements of the level (ft) of Lake Huron from 1875 to 1972, shown in Figure 2. With this example we show how to: build a basic state-space structure, specify a prior on γ, run the MCMC algorithm, plot the estimated quantiles and forecast distributions.

To estimate the dynamic distribution of the data, we consider three quantiles, $p_0 = 0.95, 0.50,$ and 0.05. We begin by creating the state-space structure and prior used to estimate the quantiles (i.e. $\mathbf{F}_t, \mathbf{G}_t, \mathbf{m}_0$, and $\mathbf{C}_0$ discussed in Section 2). We choose to model the quantiles with a second order polynomial trend, which we construct with the `polytrendMod()` function.

```
R> model = polytrendMod(order = 2, m0 = c(mean(LakeHuron),0), C0 = 10*diag(2))
R> model

Prior mean of the state vector:
$m0
         [,1]
[1,] 579.0041
[2,]   0.0000


Prior covariance of the state vector:
$C0
     [,1] [,2]
[1,]   10    0
[2,]    0   10


Observational vector:
$FF
     [,1]
[1,]    1
[2,]    0


Evolution matrix:
$GG
     [,1] [,2]
[1,]    1    1
[2,]    0    1
```

To allow variability in the dynamic quantile, we select a single discount factor of 0.9 to define the evolution covariance matrix $\mathbf{W}_t$. Discount factors can also be optimized, an example of which we will show in Section 4.2. Further, We place truncated Student-t priors on the skewness parameters via `PriorGamma` as discussed in Section 2.1. For the tracked standard-profile run, we use `n.burn = 2000` and `n.mcmc = 3000`. The R output is shown for only the last quantile using the input `verbose = TRUE`.

```
R> M95 = exdqlmMCMC(y = LakeHuron, p0 = 0.95, model = model,
+                 df = 0.9, dim.df = 2,
+                 PriorGamma = list(m_gam = -1, s_gam = 0.1, df_gam = 1),
+                 n.burn = 2000, n.mcmc = 3000, verbose = FALSE)
R> M50 = exdqlmMCMC(y = LakeHuron, p0 = 0.50, model = model,
```

```
+                   df = 0.9, dim.df = 2,
+                   PriorGamma = list(m_gam = 0, s_gam = 0.1, df_gam = 1),
+                   n.burn = 2000, n.mcmc = 3000, verbose = FALSE)
R> M5 = exdqlmMCMC(y = LakeHuron, p0 = 0.05, model = model,
+                  df = 0.9, dim.df = 2,
+                  PriorGamma = list(m_gam = 1, s_gam = 0.1, df_gam = 1),
+                  n.burn = 2000, n.mcmc = 3000, verbose = TRUE)
```

Representative console excerpt, abridged to the fields most relevant for the progress format:

```
MCMC backend: C++ (mode=fast)
MCMC init: running LDVB warm start
MCMC start | burn=2000 | keep=3000 | kernel=slice | warm_start=ldvb
...
MCMC done | runtime_sec=122.179 | sigma=0.220 | gamma=5.013 | kernel=slice
```

For this time series of length 98, the tracked standard-profile free-σ run takes about 122.2 seconds to sample 5000 iterations, as seen in the output above. By default `exdqlmMCMC()` uses the LDVB routine to warm-start the chain, and this warm-start choice can be changed independently of the subsequent MCMC kernel. Under the default setting `mh.proposal = "slice"`, the console output reports compact burn-in and retained-sample progress lines for the current values of σ and γ. For the joint random-walk kernels, the proposal covariance used to sample σ and γ can be supplied manually through `Sig.mh`; alternatively, with the setting `joint.sample = TRUE`, the covariance matrix can be recalculated from the burned sample.

For joint random-walk kernels, the same progress format also reports the current acceptance rate, but this is not applicable under the default slice update. In the targeted support comparison we use to assess this kernel choice for the free-σ Lake Huron median fit, four LDVB-warm-started chains show that the default `slice` update is more stable than `laplace_rw`: the pooled multichain summaries give $\hat{R}$ values of about 1.01 for σ and 1.04 for γ under `slice`, versus about 1.09 and 1.16 under `laplace_rw`. The corresponding effective sample sizes are about 940 and 165 under `slice`, compared with about 129 and 38 under `laplace_rw`, although `laplace_rw` is roughly twice as fast in this support run. We therefore retain `slice` as the default update in this example. The data are relatively symmetric, therefore we do not expect the skewness parameter γ to be far from zero for $p_0 = 0.5$. To examine this, we use a dedicated median run with free σ under the default `slice` update and thin the retained samples by 10 only for plotting. In that run, the posterior mean of σ is about 0.375 with a 95% credible interval $(0.298, 0.473)$, while the posterior mean of γ is -0.044 with a 95% credible interval $(-0.375, 0.321)$. All MCMC samples in the output of `exdqlmMCMC` are `mcmc` objects. Thus we can use the package **coda** to examine the trace and density plots of both σ and γ, seen in Figure 1.

```
R> M50.trace = exdqlmMCMC(y = LakeHuron, p0 = 0.50, model = model,
+                   df = 0.9, dim.df = 2,
+                   PriorGamma = list(m_gam = 0, s_gam = 0.1, df_gam = 1),
+                   n.burn = 2000, n.mcmc = 3000, verbose = FALSE)
R> keep.idx = seq(1, length(M50.trace$samp.sigma), by = 10)
R> sigma.trace = coda::mcmc(M50.trace$samp.sigma[keep.idx], thin = 10)
R> gamma.trace = coda::mcmc(M50.trace$samp.gamma[keep.idx], thin = 10)
R> coda::traceplot(sigma.trace, main = "sigma trace")
R> coda::densplot(sigma.trace, main = "sigma density")
```

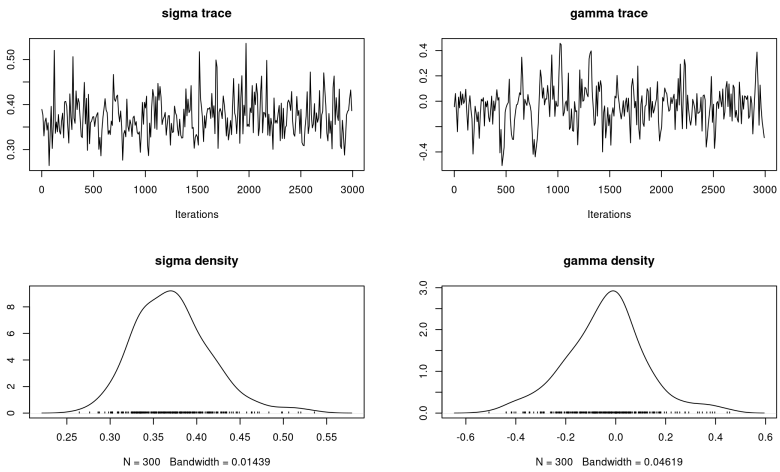

Figure 1: Example 1: Lake Huron. Trace and density plots from the dedicated free-σ median run with 2000 burn-in iterations and 3000 retained draws. For readability, every 10th retained draw is used in the plotting step. The top row shows the posterior trace plots for σ and γ, and the bottom row shows the corresponding posterior densities.

```
R> coda::traceplot(gamma.trace, main = "gamma trace")
R> coda::densplot(gamma.trace, main = "gamma density")
```

Clearly γ is indistinct from zero, thus the added flexibility of the skewness parameter is not necessary for the median. We re-run the model with a point-mass prior on γ at 0 using the settings `gam.init = 0` and `fix.gamma = TRUE`, which is equivalent to the setting `dqlm.ind = TRUE`. This reduces the model to the DQLM mentioned in Section 2.5.

```
R> M50 = exdqlmMCMC(y = LakeHuron, p0 = 0.50, model = model,
+                   df = 0.9, dim.df = 2,
+                   gam.init = 0, fix.gamma = TRUE,
+                   n.burn = 2000, n.mcmc = 3000, verbose = TRUE)

MCMC backend: C++ (mode=fast)
MCMC init: running LDVB warm start
MCMC start | burn=2000 | keep=3000 | kernel=conjugate | warm_start=ldvb
...
MCMC done | runtime_sec=51.838 | sigma=0.430 | kernel=conjugate
```

The results are `exdqlmMCMC` objects that can be used for analysis and forecasting. First we examine the estimated quantiles with the plot method for `exdqlmMCMC` objects, seen in Figure 2. This generates a plot of the data with the MAP estimates and the 95% CrIs of the dynamic quantile. Subsequent quantiles (i.e. $p_0 = 0.50, 0.05$ in this example) can be added to the plot using the function `exdqlmPlot()` with the argument `add = TRUE`.

```
R> plot(M95)
R> exdqlmPlot(M50, add = TRUE, col = "blue")
R> exdqlmPlot(M5, add = TRUE, col = "forest green")
R> legend("topright", lty = 1, col = c("purple","blue","forest green"),
+        legend = c(expression('p'[0]*'=0.95'),expression('p'[0]*'=0.50'),
+                   expression('p'[0]*'=0.05')))
```

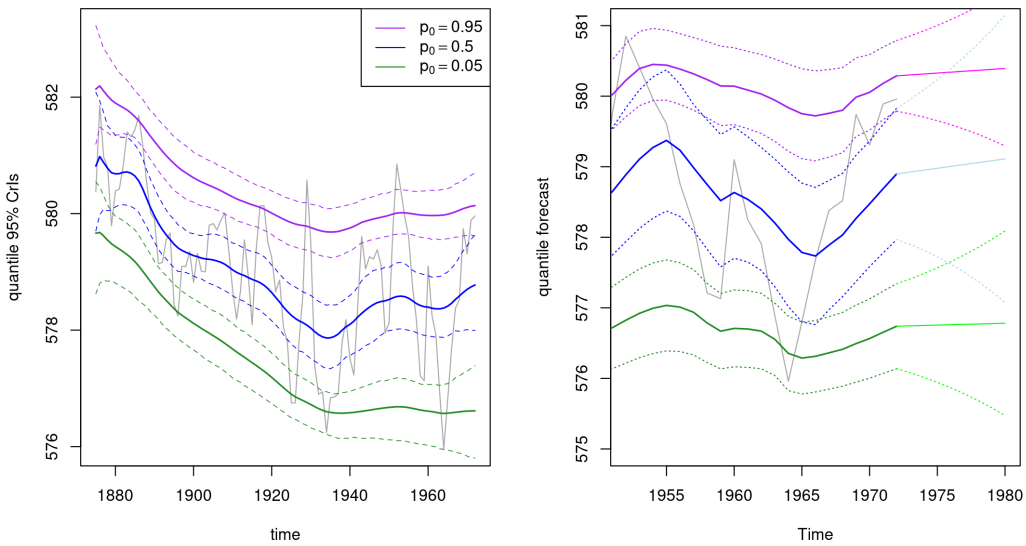

Figure 2: Example 1: Lake Huron. Left: MAP estimates and 95% CrIs of the estimated quantiles, plotted with the data (grey). Right: Forecasted quantile estimates and 95% credible intervals (seen after 1972), along with the filtered quantile estimates and 95% credible intervals (seen from 1952 to 1972).

Next, we forecast the $k = 8$ step-ahead distributions starting in 1972. To do this, we first create the observational vector (`fFF`) and evolution matrix (`fGG`) that will be used in the forecast updates. In this case both are non-time-varying and identical to those used to estimate the quantile.

```
R> fFF = model$FF
R> fGG =  model$GG
```

We plot the time series data in a narrower range for closer examination and add the forecast estimates using `exdqlmForecast()` with argument `add = TRUE`. Notice we start the forecast at the last time index of the data, i.e. `start.t = length(LakeHuron)`.

```
R> plot(LakeHuron,  xlim = c(1952,1980), ylim = c(575,581),
+        col = "dark grey")
R> exdqlmForecast(start.t = length(LakeHuron), k = 8, m1 = M95,
+               fFF = fFF, fGG = fGG, plot = TRUE, add = TRUE)
R> exdqlmForecast(start.t = length(LakeHuron), k = 8, m1 = M50,
+               fFF = fFF, fGG = fGG, plot = TRUE, add = TRUE,
+               cols = c("blue","light blue"))
R> exdqlmForecast(start.t = length(LakeHuron), k = 8, m1 = M5,
+               fFF = fFF, fGG = fGG, plot = TRUE, add = TRUE,
+               cols = c("forest green","green"))
```

This generates a plot of the data with the forecasted quantile estimates and 95% credible intervals (seen after 1972), along with the filtered quantile estimates and 95% credible intervals

(before 1972), for reference. Results can be seen in Figure 2. The percentage in the CrIs can be modified with the parameter `cr.percent`.

4.2. Sunspots

For our next example, we will use the yearly Sunspot time series from the package **datasets**. The data are yearly counts for sunspots from 1700 to 1988, shown in Figure 3. With this example we will show how to: use the **dlm** package to create the state-space model, combine blocks of a state-space model, apply the LDVB algorithm, perform visual model diagnostics to compare the exDQLM with the DQLM, and use those diagnostics for model selection.

The solar cycle impacts astronauts in space as well as lives and technology on earth. Of particular interest to scientists is the solar max, or period in which the sun is most active Fox (2012), therefore we will explore the 0.85 quantile in this example. Again, we begin by building the state-space structure used to estimate the quantile. Here we choose to model the quantile with a first order polynomial trend component as well as a Fourier form seasonal component that incorporates a fundamental period every 11 observations (years), as well as some of its corresponding harmonics. This results in a dynamic model with a total of 9 state parameters, 1 for the first order polynomial component and 8 for the seasonal component. The **exdqlm** functions are compatible with time-invariant `dlm` objects from the **dlm** package. For example, we create the first order trend component with the function `dlmModPoly()` from the **dlm** package, then use `as.exdqlm()` to turn the output into an `exdqlm` object.

```
R> dlm.trend.comp = dlm::dlmModPoly(1, m0 = mean(sunspot.year), C0 = 10)
R> trend.comp = as.exdqlm(dlm.trend.comp)
```

Next, we construct the seasonal component with our `seasMod` function. The duration of sunspot cycles typically follow a period of 11 years, of which we include the first 4 harmonics in the component.

```
R> seas.comp = seasMod(p = 11, h = 1:4, C0 = 10*diag(8))
```

To combine the two components into a single state-space structure, we add the two `exdqlm` objects.

```
R> model = trend.comp + seas.comp
```

The resulting evolution matrix of the combined models is printed for illustration.

```
R> model$GG
```

```
     [,1]    [,2]   [,3]    [,4]   [,5]    [,6]    [,7]    [,8]    [,9]
 [1,]    1  0.0000 0.0000  0.0000 0.0000  0.0000  0.0000  0.0000  0.0000
 [2,]    0  0.8413 0.5406  0.0000 0.0000  0.0000  0.0000  0.0000  0.0000
 [3,]    0 -0.5406 0.8413  0.0000 0.0000  0.0000  0.0000  0.0000  0.0000
 [4,]    0  0.0000 0.0000  0.4154 0.9096  0.0000  0.0000  0.0000  0.0000
 [5,]    0  0.0000 0.0000 -0.9096 0.4154  0.0000  0.0000  0.0000  0.0000
 [6,]    0  0.0000 0.0000  0.0000 0.0000 -0.1423  0.9898  0.0000  0.0000
 [7,]    0  0.0000 0.0000  0.0000 0.0000 -0.9898 -0.1423  0.0000  0.0000
 [8,]    0  0.0000 0.0000  0.0000 0.0000  0.0000  0.0000 -0.6549  0.7557
 [9,]    0  0.0000 0.0000  0.0000 0.0000  0.0000  0.0000 -0.7557 -0.6549
```

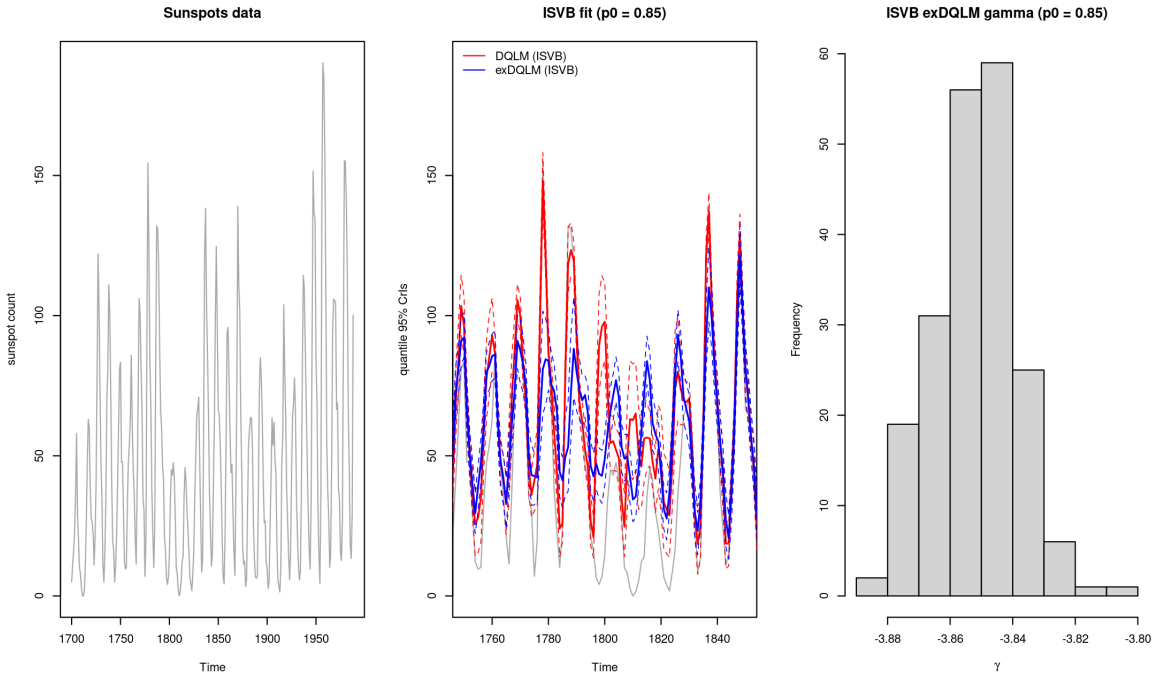

Figure 3: Example 2: Sunspots. Left: The sunspot time series from 1700 to 1988. Center: MAP estimates and 95% CrIs of the estimated quantiles from the DQLM (red) and exDQLM (blue), plotted with the data (grey) from 1750 to 1850. Right: Histogram of samples from the approximated posterior distribution of γ.

We will apply discount factors by component, i.e. a discount factor of 0.9 for the trend component of dimension 1 and a discount factor of 0.85 for the seasonal component of dimension 8. This discounting strategy can be applied with arguments `df = c(0.9,0.85)` and `dim.df = c(1,8)`.

We call the function `exdqlmLDVB()` to apply the LDVB algorithm. We use the algorithm with argument `dqlm.ind = TRUE` to estimate the DQLM, as well as with the default settings to estimate the exDQLM. The output is suppressed using input `verbose = FALSE`, however we check the run-time of each model after convergence using the `summary()` generic function. For this dataset of length 289, the LDVB algorithm takes 146 iterations and approximately 5 seconds to converge for the DQLM. Alternatively the exDQLM takes 200 iterations and approximately 31 seconds to converge.

```
R> M1 = exdqlmLDVB(y = sunspot.year, p0 = 0.85, model = model,
+                  df = c(0.9,0.85), dim.df = c(1,8),
+                  dqlm.ind = TRUE, fix.sigma = FALSE,
+                  verbose = FALSE)
R> M2 = exdqlmLDVB(y = sunspot.year, p0 = 0.85, model = model,
+                  df = c(0.9,0.85), dim.df = c(1,8),
+                  fix.sigma = FALSE,
+                  verbose = FALSE)
R> summary(M1)

Bayesian Dynamic Quantile Regression Model (DQLM)
```

```
Number of Observations: 289
State Dimension: 9
Discount factors ( dimensions ): 0.9 ( 1 ), 0.85 ( 8 )

DQLM fitted using VB
Variational Parameters: sigma
Iterations until convergence: 146
Run-time: 4.315 seconds
```

```
R> summary(M2)
```

```
Bayesian Dynamic Quantile Regression Model (exDQLM)
Number of Observations: 289
State Dimension: 9
Discount factors ( dimensions ): 0.9 ( 1 ), 0.85 ( 8 )

exDQLM fitted using LDVB
Variational Parameters: gamma sigma
Iterations until convergence: 200
Run-time: 30.571 seconds
```

We can visualize the differences between the two resulting dynamic quantiles using the function `exdqlmPlot()`, seen in Figure 3. For clarity we limit the figure to years 1750 to 1850.

```
R> plot(sunspot.year, xlim = c(1750,1850), col = "dark grey",
+       ylab = "quantile 95% CrIs")
R> exdqlmPlot(M1, add = TRUE, col = "red")
R> exdqlmPlot(M2, add = TRUE, col = "blue")
```

To examine whether the added flexibility of the exDQLM is significant, we can also visualize the approximate posterior distribution of the skewness parameter γ by plotting the samples from the corresponding variational distribution, also seen in Figure 3.

```
R> hist(M2$samp.gamma,xlab=expression(gamma),main="")
```

The approximate distribution is clearly distinct from zero, thus we expect a better model fit from the exDQLM.

To further examine the two models, we use the function `exdqlmDiagnostics()` to plot the model diagnostics discussed in Section 3. The results are seen in Figure 4. The QQ-plot and the ACF of the standard forecast errors both favor the exDQLM.

```
R> exdqlmDiagnostics(M1, M2, cols = c("red","blue"))
```

Finally, the function `exdqlmDiagnostics` can also be used for model selection. Say we want to check whether the discount factor we used for the seasonal component was optimal with respect to the KL divergence of the one-step-ahead predictive distributions. We can create a list of possible discount factors, quickly run the LDVB for each possible model, and select the set of discount factors which results in the lowest KL divergence. A simple example is seen below, which takes approximately 6 minutes to run.

```
R> possible.dfs = cbind(0.9,seq(0.85,1,0.05))
R> possible.dfs
```

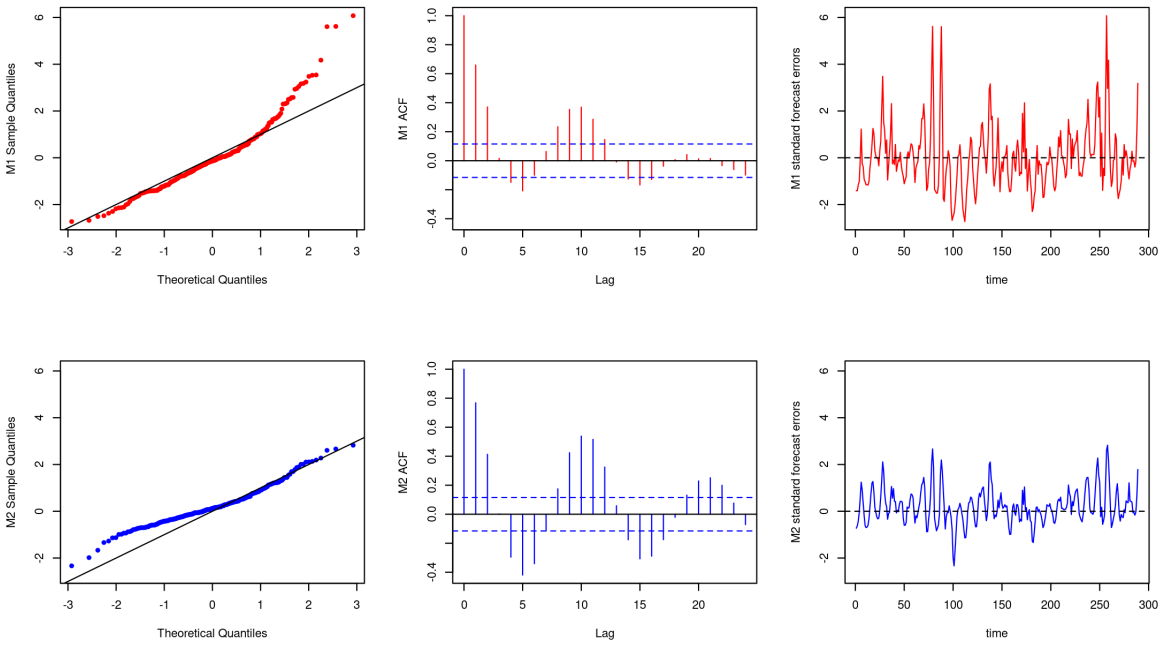

Figure 4: Example 2: Sunspots. The QQ-plots (left column), ACF plots (center column) and standard forecast errors (right column) from the DQLM (red) and exDQLM (blue).

```
     [,1] [,2]
[1,]  0.9 0.85
[2,]  0.9 0.90
[3,]  0.9 0.95
[4,]  0.9 1.00

R> KLs <- vector("numeric")
R> ref.samp = rnorm(length(sunspot.year))
R> for(i in 1:nrow(possible.dfs)){
+   temp.M2 = exdqlmLDVB(y = sunspot.year, p0 = 0.85, model = model,
+                   df = possible.dfs[i,], dim.df = c(1,8),
+                   sig.init = 2, verbose = FALSE)
+   temp.check = exdqlmDiagnostics(temp.M2, plot = FALSE,
+                               ref = ref.samp)
+   KLs = c(KLs,temp.check$m1.KL)
+ }
R> # optimal dfs based off KL divergence
R> possible.dfs[which.min(KLs),]

[1] 0.90 0.85
```

The results suggest the discount factor we used for the seasonal component is optimal.

4.3. Big Tree water flows

For our final example, we consider the average monthly natural water flow (cubic feet per second) at the Big Tree gauge of the San Lorenzo river in Santa Cruz County, CA from January

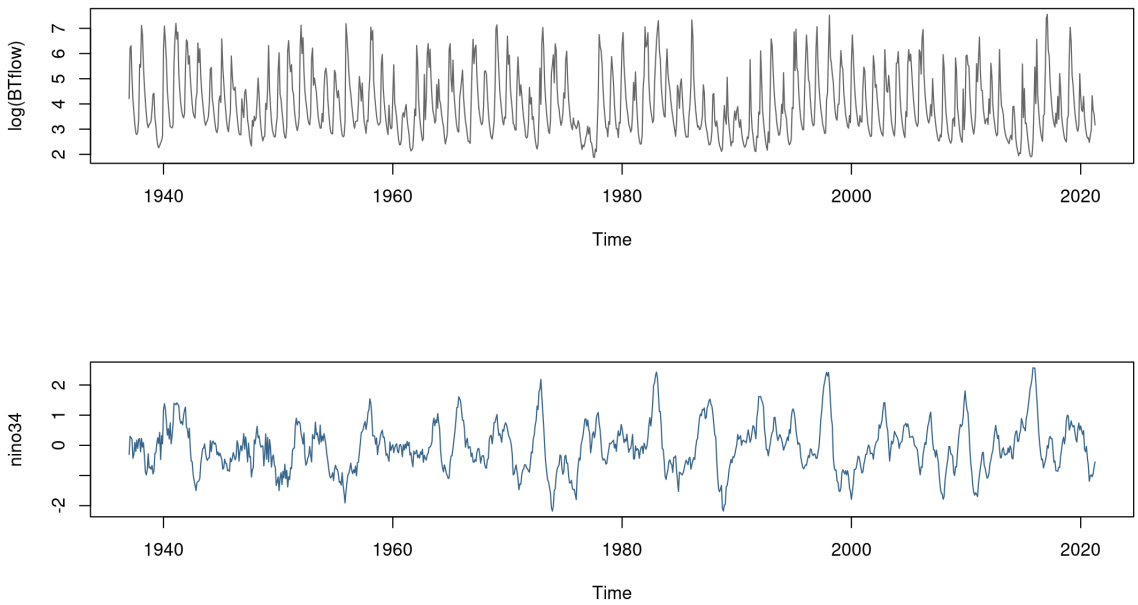

Figure 5: Example 3: Big Tree water flow. Top: Average monthly water flow at the Big Tree gauge of the San Lorenzo river in Santa Cruz County, CA, plotted on the log-scale. Bottom: Monthly values of Niño 3.4. Both time series span January 1937 through April 2021.

1937 through April 2021 (U.S. Geological Survey 2016), for a total of 1012 observations. The dataset is included in our package as `BTflow`. With this example we show how to build a state-space structure manually (particularly for a dynamic quantile regression), apply our transfer function exDQLM, examine individual components of the estimated quantiles, and perform quantitative model diagnostics to compare models.

For this particular dataset, the interest is in low quantiles as a possible indicator of drought, thus we will consider the $p_0 = 0.15$ quantile. To investigate any climatological influence on this low quantile, we include the Niño 3.4 index (Rayner, Parker, Horton, Folland, Alexander, Rowell, Kent, and Kaplan 2003) as in input of our exDQLM. The Niño 3.4 index is a measure of how anomalously warm or cool the average sea surface temperature is in the Niño 3.4 region (5°S-5°N, 170°W-120°W) of the Pacific Ocean. It is one of the most commonly used indices to define El Niño and La Niña events. The monthly Niño 3.4 dataset used in our analysis is also included in the package as `nino34`. We model the relationship between Niño 3.4 and the quantile with two model structures: a dynamic quantile regression and a transfer function as seen in Section 2.4. To avoid modeling problems caused by the scale and extremes in the Big Tree dataset, we fit the following models to the time series on the log scale seen in Figure 5.

In addition to the components that capture the relationship with the Niño 3.4 index, we also include a first order polynomial trend component as well as a Fourier form seasonal component that includes a single periodic component with period 12. The Big Tree water flow time series follows a period of twelve months, of which we include only the first (annual) harmonic.

```
R> trend.comp = polytrendMod(1, m0 = 3, C0 = 0.1)
```

```
R> seas.comp = seasMod(p = 12, h = 1, C0 = diag(1,2))
R> model = trend.comp + seas.comp
```

Next, we create the dynamic regression component for our first model using `regMod()`. Combining this with the trend and seasonal component we have our complete state-space structure for the dynamic quantile regression model.

```
R> reg.comp = regMod(nino34, m0 = 1, C0 = 1)
R> model.w.reg = model + reg.comp
```

To allow the Niño 3.4 input components of both models to vary over time, we set the respective discount factors to be 0.95 (this is done via the parameter `df` in `exdqlmLDVB()` and the parameter `tf.df` in `transfn_exdqlmLDVB()`). Alternatively, we set the discount factors to 1 and 0.9 for the trend and seasonal components, respectively. This allows the seasonal amplitudes to evolve over time, while any further variation in the 0.15 quantile from year to year will solely be attributed to the effects of the Niño 3.4 input in our models.

The transfer function model is created within the function `transfn_exdqlmLDVB()` and follows the structure detailed in Section 2.4. Optimization using the KL divergence suggests the effects of the Niño 3.4 index decay at an optimal transfer function rate of $\lambda = 0.85$.

```
R> possible.lams = seq(0.1,0.9,0.05)
R> KLs <- vector("numeric")
R> ref.samp = rnorm(length(BTflow))
R> for(i in 1:length(possible.lams)){
R>   temp.M2 = transfn_exdqlmLDVB(y = log(BTflow), p0 = 0.15, model = model,
+                                df = c(1,0.9), dim.df = c(1,2),
+                                X = nino34, tf.df = c(0.95), lam = possible.lams[i],
+                                tf.m0 = c(0,0), tf.C0 = diag(c(0.1,0.005),2),
+                                sig.init = 0.1, gam.init = - 0.1,
+                                tol = 0.05, verbose = FALSE)
R>   temp.check = exdqlmDiagnostics(temp.M2, plot = FALSE,
+                            ref = ref.samp)
R>   KLs = c(KLs,temp.check$m1.KL)
R> }
R> # optimal lambda based off KL divergence
R> possible.lams[which.min(KLs)]
```

```
[1] 0.85
```

With these discount factors and optimal λ, we estimate the dynamic quantile regression model (`M1`) and transfer function model (`M2`).

```
R> M1 = exdqlmLDVB(y = log(BTflow), p0 = 0.15, model = model.w.reg,
+                df = c(1,0.9,0.95), dim.df = c(1,2,1),
+                sig.init = 0.1, gam.init = - 0.1,
+                tol = 0.05, verbose = FALSE)
R> M2 = transfn_exdqlmLDVB(y = log(BTflow), p0 = 0.15, model = model,
+                         df = c(1,0.9), dim.df = c(1,2),
+                         X = nino34, tf.df = c(0.95), lam = 0.85,
+                         tf.m0 = c(0,0), tf.C0 = diag(c(0.1,0.005),2),
+                         sig.init = 0.1, gam.init = - 0.1,
+                         tol = 0.05, verbose = FALSE)
```

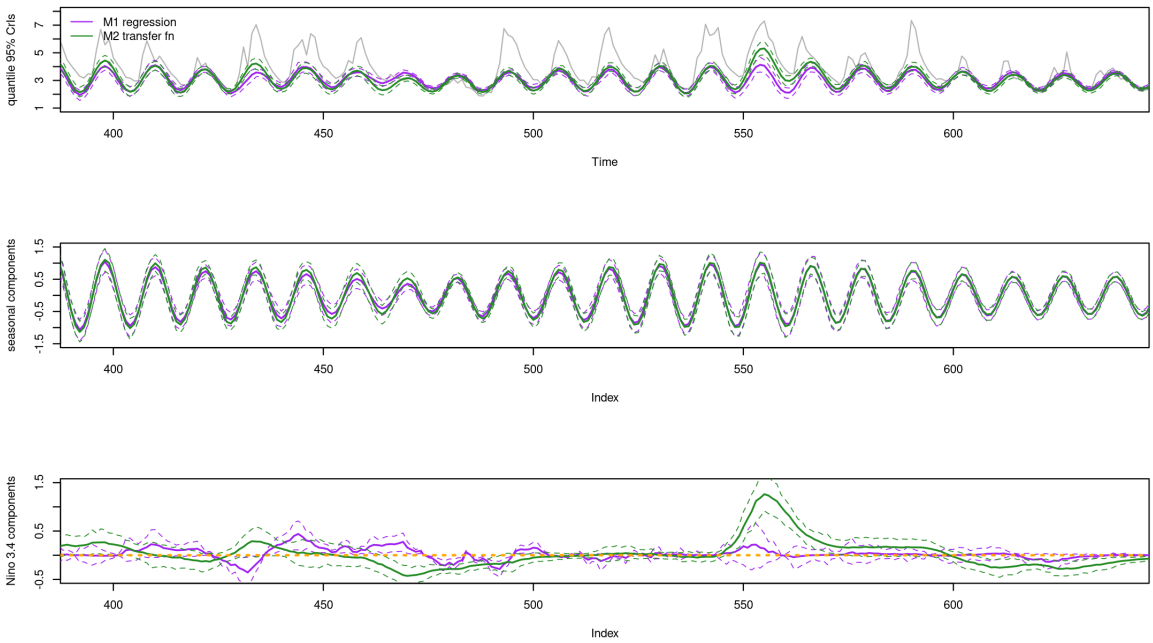

Figure 6: Example 3: Big Tree water flow. Top: MAP estimates and 95% CrIs of the estimated quantiles from the models with the regression component (purple) and transfer function component (green), plotted with the data (grey) from 1970 to 1990. Middle: MAP estimates and 95% CrIs of the annual seasonal components. Bottom: MAP estimates and 95% CrIs of the components which model the quantile association with Niño 3.4. The estimates of the dynamic regression component $X_t\boldsymbol{\theta}_{4,t}$ of M_1 where $\boldsymbol{\theta}_{4,t}$ denotes the fourth element of the state vector, are seen in purple. The estimates of the transfer function component ζ_t of M_2 are in green. The horizontal orange dotted line is at zero for reference.

To visualize the differences between the two resulting dynamic quantiles we use the function `exdqlmPlot`, results of which are found in Figure 6. For clarity we limit the figure to view 1970 to 1990.

```
R> plot(log(BTflow), col = "grey", ylim = c(1,8), xlim=c(1970,1990),
+       ylab = "quantile 95% CrIs")
R> exdqlmPlot(M1, add = TRUE)
R> exdqlmPlot(M2, add = TRUE, col = "forest green")
```

Clearly the resulting quantiles have significant differences. To identify from where these differences arise, we can utilize the function `compPlot` to examine the components of the models more closely. We examine the annual component of the parameter vector first. The annual component is comprised of the second and third elements of the parameter vector $\boldsymbol{\theta}_t$, thus we set the parameter `index = c(2,3)` to visualize this component. The result is seen in Figure 6, again limited to the years 1970 to 1990.

```
R> plot(NA, ylim = c(-1.5,1.5), xlim = c(1970,1990),
+         ylab = "seasonal components")
```

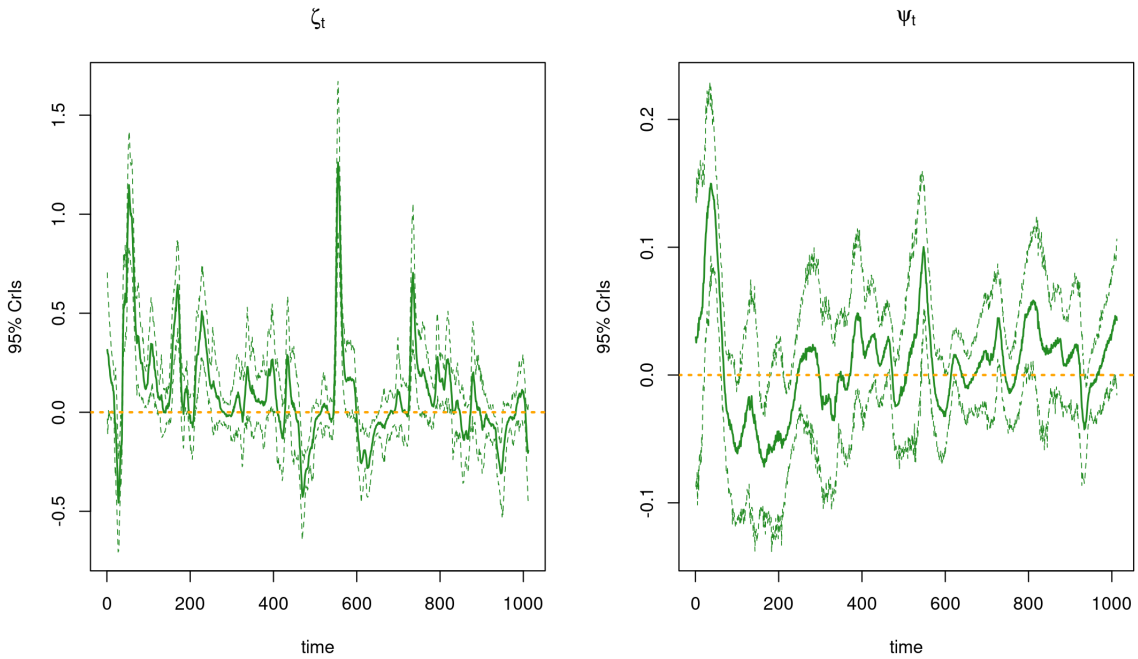

Figure 7: Example 3: Big Tree water flow. MAP estimates and 95% CrIs of parameters ζ_t and ψ_t from the transfer function component. Horizontal dotted orange lines are at zero for reference.

```
R> compPlot(M1, index = c(2,3), add = TRUE)
R> compPlot(M2, index = c(2,3), add = TRUE, col = "forest green")
```

Although the seasonal components of the two models exhibit differences in amplitude, we expect more significant differences to appear in the Niño 3.4 component of each model. Again, we use the function `compPlot` to visualize these components seen in Figure 6 from 1970 to 1990.

```
R> plot(NA, ylim = c(-0.5,1.5), xlim = c(1970,1990),
+          ylab = "NiÃśo 3.4 components")
R> compPlot(M1, index = c(4), add = TRUE)
R> compPlot(M2, index = c(4,5), add = TRUE, col = "forest green")
R> abline(h = 0, col = "orange", lty = 3, lwd = 2)
```

For the transfer function model, `M2`, it is also of interest to look at the estimates of parameters ζ_t and ψ_t directly. Because these parameters augmented the state-space model as discussed in Section 2.4, the function `compPlot` can also be used to look at these estimates directly using the parameter `just.theta = TRUE`. This setting returns the dynamic posterior estimates of a single index of the state vector $\boldsymbol{\theta}_t$ (or $\tilde{\boldsymbol{\theta}}_t$ in the case of the transfer function model). Parameter `index` of `4` and `5` correspond to ζ_t and ψ_t, respectively. Results can be found in Figure 7.

```
R> compPlot(M2, index = 4, col= "forest green",
+             add = FALSE, just.theta = TRUE)
R> abline(h = 0, col = "orange", lty = 3, lwd = 2)
```

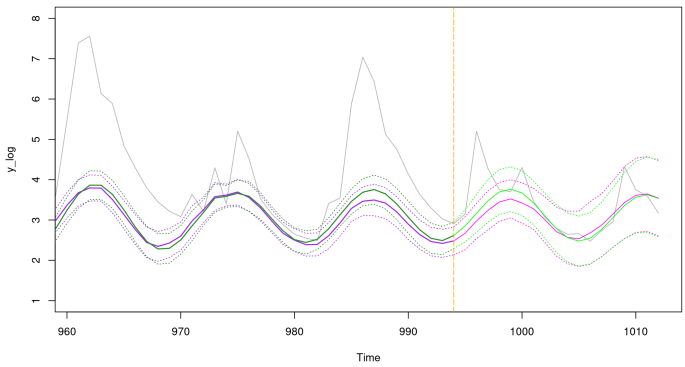

Figure 8: Example 3: Big Tree water flow. 18-step-ahead quantile forecast beginning October 2019 through April 2021 overlaid on the log Big Tree water flow time series. The vertical dot-dashed line is at the beginning of the forecast period, October 2019, for reference. Solid lines indicate mean estimates and dotted lines 95% CrI, with the filtered estimates seen leading up to October 2019 and the forecast estimates beyond October 2019. Estimates from M_1, the quantile regression model, are seen in purple/pink. Estimates from M_2, the transfer function model, are seen in green.

```
R> title(expression(eta[t]))
R> compPlot(M2, index = 5, col = "forest green",
+             add = FALSE, just.theta = TRUE)
R> abline(h = 0, col = "orange", lty = 3, lwd = 2)
R> title(expression(psi[t]))
```

To further understand these transfer function parameters, we examine the median number of time steps until the effect of Niño 3.4 on the 0.15 quantile is less than or equal to 1e−3, as discussed in Section 2.4. A median value 20.76 suggests the effect of the Niño 3.4 index value at time t last for around 21 months.

```
R> M2$median.kt

[1] 20.76305
```

This suggests a long-term forecast could be of interest. Thus we forecast the $k = 18$ step ahead distributions for both models starting in October 2019, results of which are seen in Figure 8.

```
R> exdqlmForecast(start.t = length(BTflow)-18, k = 18, m1 = M1)
R> exdqlmForecast(start.t = length(BTflow)-18, k = 18, m1 = M2,
+                   add = TRUE, cols = c("forest green","green"))
```

Finally, we use the function `exdqlmDiagnostics` to compute the KL divergence and pplc discussed in Section 3, results of which are seen in Table 6.

```
R> checks.out = exdqlmDiagnostics(M1, M2,
+                              cols = c("purple","orange"), plot = FALSE)
```

	M1		M2	
	output	value	output	value
pplc	m1.pplc	957.72	m2.pplc	897.22
KL	m1.kl	1.135	m2.kl	0.880

Table 6: Example 3: Big Tree water flow. Model diagnostic output from exdqlmDiagnostics() and resulting values.

The KL divergence and pplc both clearly favor the nonlinear relationship captured by the transfer function model.

4.4. Static exAL regression on a simulated heteroskedastic example

To illustrate the static routines under sparse shrinkage, we consider a correlated Gaussian regression benchmark for which the target conditional quantile is known exactly. Let

$$\mathbf{x}_i \sim N_8(\mathbf{0}, \Sigma_x), \qquad (\Sigma_x)_{jk} = 0.5^{|j-k|},$$

with sparse signal

$$\boldsymbol{\beta}^\star = (3,\ 1.5,\ 0,\ 0,\ 2,\ 0,\ 0,\ 0)^\top.$$

For each target quantile level p_0, we generate

$$Y_i^{(p_0)} = \mathbf{x}_i^\top \boldsymbol{\beta}^\star + \sigma_\varepsilon\{Z_i - \Phi^{-1}(p_0)\}, \qquad Z_i \sim N(0,1), \qquad \sigma_\varepsilon = 1.5.$$

Then

$$Q_Y(p_0 \mid \mathbf{x}_i) = \mathbf{x}_i^\top \boldsymbol{\beta}^\star,$$

so the true p_0-quantile is the same sparse linear signal at each fitted quantile level. We use $n = 160$ training observations and an independent holdout sample of size 800, and fit separate static exAL models at $p_0 \in \{0.05, 0.25, 0.50\}$. Both fits use the regularized horseshoe prior of Nishimura and Suchard (2023) via `beta_prior = "rhs_ns"` with $\tau_0 = 0.15$, fixed slab $\zeta^2 = 9$, and an unshrunk intercept. We fit `exal_static_LDVB()` and `exal_static_mcmc()`; the MCMC routine uses its default `slice` kernel and is warm-started from the LDVB fit. The lower-tail LDVB fit at $p_0 = 0.05$ converges more slowly, so we allow a larger LDVB iteration budget there. Setting `dqlm.ind = TRUE` would recover the AL special case discussed in Section 2, but here we focus on the general static exAL model with sparse shrinkage.

```
R> Sigma.x = 0.5 ^ as.matrix(dist(1:8))
R> beta.true = c(3, 1.5, 0, 0, 2, 0, 0, 0)
R> rhs.ctrl = list(tau0 = 0.15, a_zeta = 2, b_zeta = 9,
                   zeta2_fixed = 9, shrink_intercept = FALSE)
R> sim.y = function(X.raw, p0) {
      drop(X.raw %*% beta.true) + 1.5 * (rnorm(nrow(X.raw)) - qnorm(p0))
    }
R> X.raw = MASS::mvrnorm(160, mu = rep(0, 8), Sigma = Sigma.x)
R> X = cbind(1, X.raw)
R> p.grid = c(0.05, 0.25, 0.50)
R> y.train = lapply(p.grid, function(p0) sim.y(X.raw, p0))
R> M.ldvb = lapply(p.grid, function(p0) exal_static_LDVB(
+                    y = y.train[[which(p.grid == p0)]], X = X, p0 = p0,
```

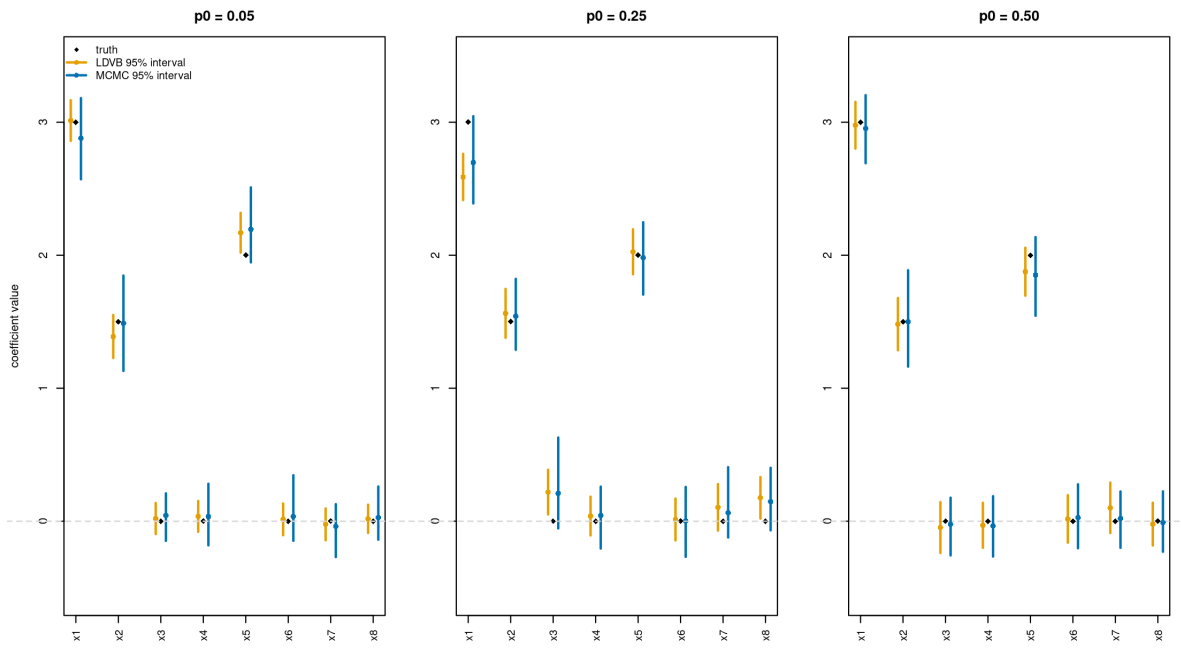

Figure 9: Example 4: sparse static exAL regression under the correlated Gaussian benchmark with the rhs_ns prior. Black diamonds denote the true non-intercept coefficients, orange intervals denote LDVB 95% posterior intervals, and blue intervals denote MCMC 95% posterior intervals. Panels correspond to p0 = 0.05, 0.25, and 0.50.

```
+                    beta_prior = "rhs_ns",
+                    beta_prior_controls = rhs.ctrl,
+                    max_iter = ifelse(p0 == 0.05, 420, 260),
+                    tol = 1e-4, verbose = FALSE))
R> M.mcmc = lapply(p.grid, function(p0) exal_static_mcmc(
+                    y = y.train[[which(p.grid == p0)]], X = X, p0 = p0,
+                    beta_prior = "rhs_ns",
+                    beta_prior_controls = rhs.ctrl,
+                    n.burn = 400, n.mcmc = 300,
+                    init.from.vb = TRUE,
+                    verbose = FALSE))
```

For static fits, the helper function `exalDiagnostics()` summarizes fitted quantiles on a common design matrix and, when a reference quantile is supplied, reports truth-recovery metrics directly. For example, on an independent holdout design:

```
R> X.hold.raw = MASS::mvrnorm(800, mu = rep(0, 8), Sigma = Sigma.x)
R> X.hold = cbind(1, X.hold.raw)
R> ref.hold = drop(X.hold %*% c(0, beta.true))
R> y.hold = sim.y(X.hold.raw, 0.25)
R> exalDiagnostics(M.ldvb[[2]], M.mcmc[[2]],
+                  X = X.hold, y = y.hold, ref = ref.hold,
+                  plot = FALSE)
```

Figure 9 shows that both methods recover the active coefficients and shrink the null coefficients

p_0	method	runtime (s)	active RMSE	null MAE	holdout quantile RMSE
0.05	LDVB	23.40	0.117	0.022	0.245
0.05	MCMC	19.12	0.133	0.036	0.274
0.25	LDVB	10.09	0.241	0.110	0.513
0.25	MCMC	18.68	0.177	0.093	0.402
0.50	LDVB	2.65	0.074	0.044	0.225
0.50	MCMC	21.02	0.090	0.023	0.248

Table 7: Example 4: runtime and sparse-signal recovery metrics for the static exAL fits under the rhs_ns prior. The active RMSE is computed over the three nonzero coefficients, the null MAE is computed over the five zero coefficients, and the holdout quantile RMSE is computed against the true target quantile on an independent holdout sample of size 800.

toward zero across the lower tail, lower-middle quantile, and median. The lower-tail fit at $p_0 = 0.05$ is the most demanding case, yet both methods recover the sparse signal cleanly; on this run, LDVB is slightly better on the active-signal RMSE and holdout quantile RMSE while still requiring the larger iteration budget. At $p_0 = 0.25$, the two fits remain qualitatively aligned, with MCMC modestly closer to the true sparse signal. At $p_0 = 0.50$, both fits again track the true sparse pattern closely, with LDVB slightly sharper on the active coefficients and holdout quantile recovery while MCMC shrinks the null coefficients more aggressively.

Table 7 reinforces the visual comparison. The two methods are competitive across all three quantile levels, but the relative advantage is not uniform: LDVB is slightly stronger in the lower-tail and median runs on the active-signal and holdout-recovery metrics, whereas MCMC is stronger at $p_0 = 0.25$ and also tends to shrink the null coefficients more aggressively. In runtime terms, the two methods are of roughly comparable cost at the hardest lower-tail fit, while LDVB is clearly faster away from that case: it is about 1.9 times faster at $p_0 = 0.25$ and about 7.9 times faster at $p_0 = 0.50$. The main message is that static LDVB can deliver substantial computational savings as a first-pass approximation, but this speed comes with the usual variational tradeoff that posterior uncertainty quantification may be less faithful than the exact MCMC baseline. Accordingly, the static `rhs_ns` workflow supports both roles cleanly: LDVB for rapid screening and model exploration, and static MCMC as the reference method when higher-fidelity posterior uncertainty quantification is required.

5. Conclusion

Our exDQLM framework provides a robust and versatile tool for dynamic quantile modeling, implemented in the presented **exdqlm** package. Relevant features of the data can easily be incorporated in the evolution of the quantile. The MCMC algorithm provides a method for exact posterior inference. Alternatively, the LDVB algorithm provides a method for fast approximate inference enabling estimation for large data sets, quick examination of relevant model features, and model selection. Further, our transfer function model provides a straightforward method for quantifying a nonlinear relationship between a quantile and response. Finally, our forecast and diagnostic routines facilitate further model exploration. These capabilities have been discussed in detail and demonstrated with a set of examples. The **exdqlm** package makes these methods, and more generally, dynamic quantile modeling, accessible to all levels of users.

References

Barata R, Prado R, Sansó B (2022). "Fast inference for time-varying quantiles via flexible dynamic models with application to the characterization of atmospheric rivers." *The annals of applied statistics*.

Beal MJ (2003). *Variational algorithms for approximate Bayesian inference*. Ph.D. thesis, UCL (University College London).

Benoit D, Al-Hamzawi R, Yu K, Van den Poel D (2011). ***bayesQR**: A Bayesian approach to quantile regression*. R package version 2.3, URL https://CRAN.R-project.org/package=bayesQR.

Carter CK, Kohn R (1994). "On Gibbs sampling for state space models." *Biometrika*, **81**(3), 541–553.

Fox K (2012). "Solar Minimum; Solar Maximum." URL https://www.nasa.gov/mission_pages/sunearth/news/solarmin-max.html.

Frühwirth-Schnatter S (1994). "Data augmentation and dynamic linear models." *Journal of time series analysis*, **15**(2), 183–202.

Gelfand AE, Ghosh SK (1998). "Model choice: a minimum posterior predictive loss approach." *Biometrika*, **85**(1), 1–11.

Gonçalves K, Migon HS, Bastos LS (2017). "Dynamic quantile linear model: a Bayesian approach." *arXiv preprint arXiv:1711.00162*.

Huerta G, Jiang W, Tanner MA (2003). "Time series modeling via hierarchical mixtures." *Statistica Sinica*, pp. 1097–1118.

Koenker R (2005). *Quantile regression*. Cambridge University Press, New York.

Koenker R (2021). ***quantreg**: Quantile Regression*. R package version 5.85, URL https://CRAN.R-project.org/package=quantreg.

Kottas A, Krnjajić M (2009). "Bayesian semiparametric modelling in quantile regression." *Scandinavian Journal of Statistics*, **36**(2), 297–319.

Kozumi H, Kobayashi G (2011). "Gibbs sampling methods for Bayesian quantile regression." *Journal of statistical computation and simulation*, **81**(11), 1565–1578.

Kullback S, Leibler RA (1951). "On information and sufficiency." *The annals of mathematical statistics*, **22**(1), 79–86.

Nishimura A, Suchard MA (2023). "Shrinkage with shrunken shoulders: Gibbs sampling shrinkage model posteriors with guaranteed convergence rates." *Bayesian Analysis*, **18**(2), 367–390. doi:10.1214/22-BA1308.

Ostwald D, Kirilina E, Starke L, Blankenburg F (2014). "A tutorial on variational Bayes for latent linear stochastic time-series models." *Journal of Mathematical Psychology*, **60**, 1–19.

Ou L, Hunter M, Chow SM, Ji L, Chen M, Hung HJ, Lee J, Li Y, Park J (2021). ***dynr**: Dynamic Models with Regime-Switching*. R package version 0.1.16-2, URL https://CRAN.R-project.org/package=dynr.

Petris G, Gilks W (2018). ***dlm**: Bayesian and Likelihood Analysis of Dynamic Linear Models*. R package version 1.1-5, URL https://CRAN.R-project.org/package=dlm.

Prado R, Molina F, Huerta G (2006). "Multivariate time series modeling and classification via hierarchical VAR mixtures." *Computational Statistics & Data Analysis*, **51**(3), 1445–1462.

R Core Team (2013). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. URL http://www.R-project.org/.

Rayner N, Parker DE, Horton E, Folland CK, Alexander LV, Rowell D, Kent EC, Kaplan A (2003). "Global analyses of sea surface temperature, sea ice, and night marine air temperature since the late nineteenth century." *Journal of Geophysical Research: Atmospheres*, **108**(D14).

Reich BJ, Bondell HD, Wang HJ (2009). "Flexible Bayesian quantile regression for independent and clustered data." *Biostatistics*, **11**(2), 337–352.

Rosenblatt M (1952). "Remarks on a multivariate transformation." *The annals of mathematical statistics*, **23**(3), 470–472.

Taddy MA, Kottas A (2010). "A Bayesian nonparametric approach to inference for quantile regression." *Journal of Business & Economic Statistics*, **28**(3), 357–369.

US Geological Survey (2016). "National Water Information System data available on the World Wide Web (USGS Water Data for the Nation)." URL http://waterdata.usgs.gov/nwis/.

Wang C, Blei DM (2013). "Variational Inference in Non-Conjugate Models." *arXiv preprint arXiv:1209.4360*. URL https://arxiv.org/abs/1209.4360.

West M, Harrison PJ, Migon HS (1985). "Dynamic generalized linear models and Bayesian forecasting." *Journal of the American Statistical Association*, **80**(389), 73–83.

Yan Y, Kottas A (2017). "A new family of error distributions for Bayesian quantile regression." *arXiv preprint arXiv:1701.05666*.

Yu K, Moyeed RA (2001). "Bayesian quantile regression." *Statistics & Probability Letters*, **54**(4), 437–447.

Affiliation:

Antonio Aguirre and Raquel Barata
Department of Statistics
University of California Santa Cruz
Santa Cruz, CA 95064
E-mail: jaguir26@ucsc.edu, raquel.a.barata@gmail.com

Journal of Statistical Software http://www.jstatsoft.org/
published by the Foundation for Open Access Statistics http://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd